

Spoofing Android GNSS measurements

Andrea Dimaggio 346375, Antonella Gaudino 364124, Martina Muscillo 364247, Emanuele Nasso 362978

1 INTRODUCTION

GNSS systems are fundamental for positioning, navigation, and timing applications. They estimate the receiver's position by measuring the propagation time of signals transmitted from at least four satellites. Despite their high reliability, these systems present vulnerabilities, particularly to spoofing attacks, in which counterfeit signals induce the receiver to compute incorrect positions, with potentially critical consequences. This work analyzes the impact of spoofing on GNSS position estimation. To this end, raw data were collected using an Android application and subsequently processed with a MATLAB script. The comparison between authentic and altered data made it possible to highlight the distortions introduced by the attack and to discuss possible detection strategies, with the aim of improving the robustness of GNSS systems.

2 TOOLS AND SET UP ENVIRONMENT

All measurements were carried out at **Camplus Torino Codegone**, on the seventh-floor terrace. During data collection, weather conditions were clear with no precipitation, although strong wind was present. Data acquisition took place between **7:00 PM and 10:00 PM**, with each session lasting **five minutes**. The following tools were used:

- **GNSS receiver (Android smartphone):** an **OPPO A16s** running **Android 13** was used to collect raw GNSS measurements, supported by the hardware capabilities of the device
- **GNSSLogger:** an open-source application developed by Google, used to gather data from multiple satellite constellations (GPS, Galileo, BeiDou, GLONASS, etc.) and **export them in .txt format for analysis**
- **Pre-configured MATLAB script:** used to process and analyze the collected data. The script computes positioning metrics and generates plots, and includes configurable parameters (*spoof.position*, *spoof.t_start*, and *spoof.delay*) to simulate spoofed positions, define the spoofing start time, and introduce artificial signal delays.

3 STUDIED CASES

The experiment starts with the acquisition of raw measurements at the true location, recorded at **08:28 PM** and used as a reference for comparison in the following scenarios. The actual coordinates (Figure 1) are:

Latitude: 45°04'00.16" N

Longitude: 7°39'22.85" E

Altitude: 271.81 m

The altitude value was taken as provided by *Google Earth*, corresponding to 250.81 m above ground level. An additional estimated 3 m per floor was then added.

Several different scenarios were examined:

- A spoofed location without any delay
- A spoofed location with a 0.1 ms delay
- Battery-saving mode enabled during measurement
- Three people connected in a triangular arrangement via hotspot to the Android device

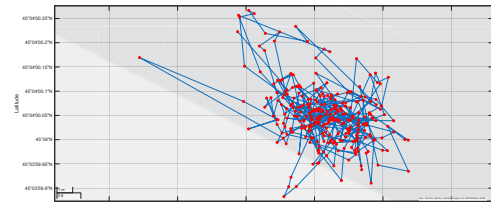


Figure 1: Actual location without spoofing.

3.1 Spoofed location without any delay

A spoofed attack was simulated using a MATLAB script, with the aim of replacing the beginning GNSS signal with a false position (45°03'59.03" N, 7°39'37.38" E, 249 m). The coordinates of the spoofed location, corresponding to Associazione Culturale Comala and located approximately 350 m from the real position, were obtained using Google Maps. The attack was set to start 150 seconds after the beginning of the measurement, and in this case no artificial delay was introduced.

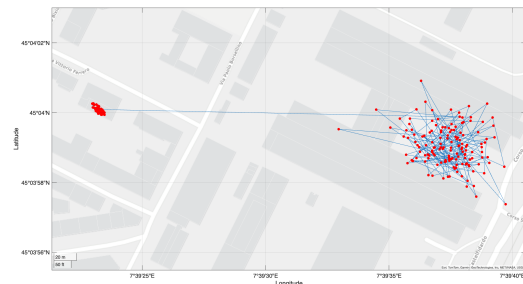


Figure 2: Location during spoofing attack simulation.

3.2 Spoofed location with 0.1 ms delay

In this scenario, an additional delay of 0.1 milliseconds was introduced between the signal transmission and reception. The aim of this test was to evaluate how even a relatively small temporal offset can affect the GNSS receiver's position estimation, as well as to investigate possible indicators for detecting anomalies caused by spoofing.

3.3 Battery-Saving Mode Interference

The GNSS data were recorded while the smartphone was set to battery-saving mode. Under this configuration, the device typically reduces background activity and limits access to certain hardware resources. The purpose of this test was therefore to investigate

whether such energy-saving settings can lead to a degradation in GNSS measurement performance, and also affect the internal clock.

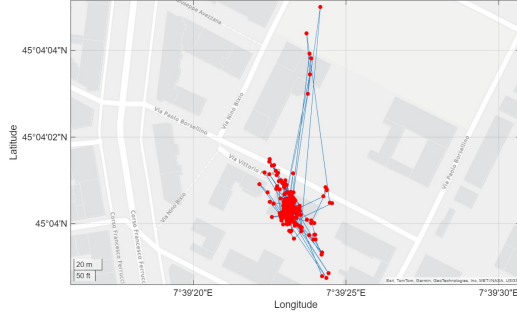


Figure 3: Location with battery-saving mode enabled.

3.4 Hotspot interference

GNSS data was collected at 9:41 PM with the Android hotspot enabled and three nearby devices connected at the vertices of a triangle. **This setup increases network activity, processor load, and RF usage, potentially causing electromagnetic interference and resource competition that may affect clock stability and signal processing.** The aim was to evaluate whether simultaneous hotspot usage degrades GNSS performance.

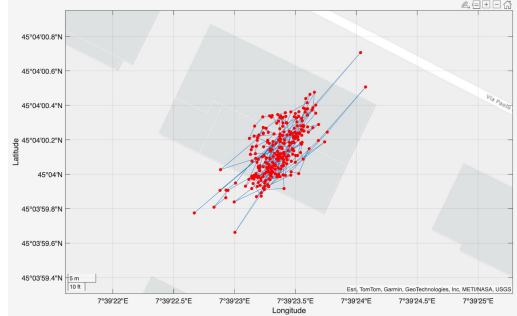


Figure 4: Location during hotspot interference.

4 RESULTS

4.1 Standard vs dataset (Task 3)

The collected data shows better overall positioning precision compared to the provided dataset. As shown in Figure 5, the measurements are more stable over time, with no evident clock discontinuities, while the provided dataset exhibits a brief irregularity that may have negatively influenced accuracy. Signal quality also appears higher, with reduced dispersion and less noise, and the spread of estimated positions is noticeably smaller. The points are more concentrated around the central value, indicating a lower variance and therefore a higher precision (Figure 6).

The nearly constant C/N_0 observed for satellite 2 (Figure 7) suggests a stable line-of-sight condition with high elevation and minimal signal obstruction, possibly due to the receiver being located on the seventh floor, which likely reduced the impact of surrounding obstacles such as nearby buildings, vegetation, or street-level clutter. Finally, although it cannot be directly verified from the plots, the

improved performance may also be influenced by better measurement conditions (e.g., fewer obstructions or open-sky environment) or a more recent GNSS receiver.



Figure 5: Clock discontinuity of dataset provided (on the left) and measured (on the right).

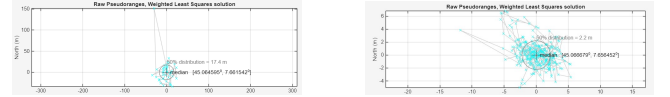


Figure 6: Computed position of data set provided (on the left) and measured (on the right)

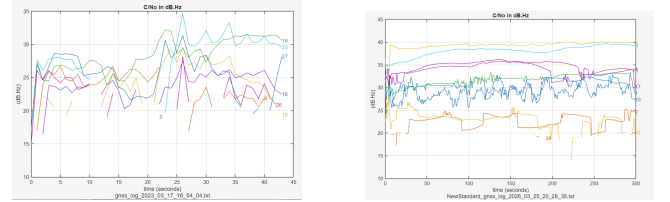


Figure 7: Signal power of dataset provided (on the left) and measured (on the right)

4.2 Spoofed location without any delay (Task 5)

After simulating the spoofing attack, results were consistent with expectations. C/N_0 values remain unchanged before and after activation, as the attack operates at the data level through post-processing rather than via counterfeit RF signals. This exposes a key limitation of signal-quality-based detection: data-level spoofing stays invisible in C/N_0 plots while still corrupting the positioning solution.

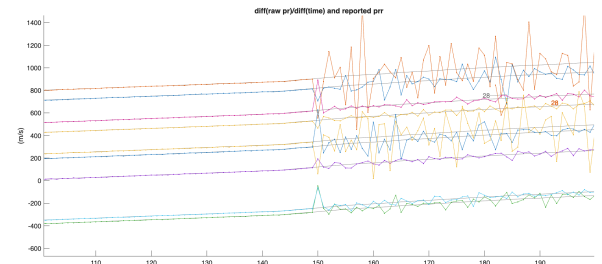


Figure 8: Rate of pseudoranges variation.

As shown in Figure 8, after $t \approx 150$ s:

- the derivative of the pseudoranges shows sharp spikes;
- the computed variation of pseudorange no longer matches the receiver-reported rate (PRR);
- the measurements become noisy, exhibiting fast and irregular fluctuations over time.

These effects are not physically plausible, as the relative motion between satellites and receiver is inherently smooth. This indicates that a violation of the GNSS measurement model has occurred.

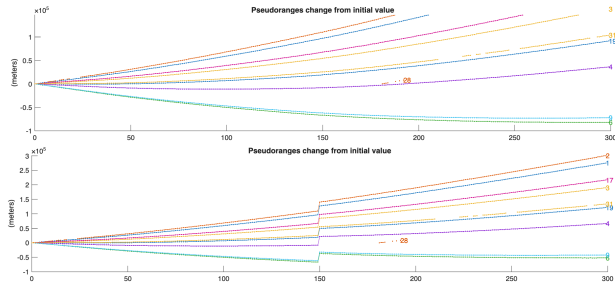


Figure 9: Pseudoranges under spoofing before vs after delay injection.

4.3 Spoofing with delay (Task 6)

As revealed in Figure 9, adding a delay introduces a sharp, simultaneous discontinuity across all satellite pseudoranges at $t \approx 150$ s which would be immediately flagged by integrity monitoring algorithms or carrier-phase smoothing. However, despite this clear anomaly in the pseudoranges, the position estimate remains largely unaffected and consistent with the no-delay spoofing case: since the receiver clock bias enters identically and additively in every pseudorange equation, the solver absorbs the spoofing-induced common shift into the clock estimate rather than translating it into a position error.

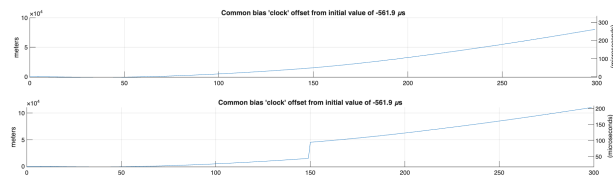


Figure 10: User clock bias under spoofing before vs after delay injection.

This is further confirmed by the comparison shown in Figure 10. In the delayed case, the common clock bias exhibits a clear discontinuity of approximately $+90 \mu\text{s}$ exactly at the spoofing onset. In contrast, the no-delay case evolves smoothly, making it the stealthier attack.

4.4 Implications of possible interference sources (Task 7)

4.4.1 Hotspot interference. The analysis focuses on the stability of the Weighted Least Squares (WLS) solution, satellite geometry, signal quality, and raw observable behavior. The standard scenario provides a stable and well-clustered solution around the median position ([45.066679, 7.656452]), with a 50% horizontal error of approximately 2.2 m and limited dispersion.

With hotspot enabled, performance degrades: the solution becomes more scattered, with a median position ([45.066699, 7.656488]) and an error of approximately 4.4 m. (Figure 11). The horizontal velocity shows larger oscillations and spikes, indicating reduced stability.

Signal quality is also impacted: the average C/N_0 decreases from approximately 29.6 dB-Hz to about 24.1 dB-Hz (-5 dB-Hz), indicating weaker and noisier signals, which lead to increased dispersion and reduced robustness of the solution. Pseudorange measurements appear more fragmented and rely on a smaller subset of satellites (Figure 12).

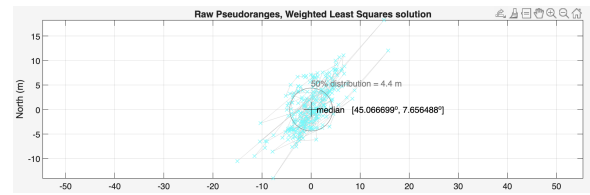


Figure 11: Median position during hotspot interference.

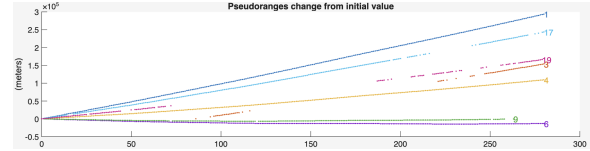


Figure 12: Pseudoranges under hotspot interference.

4.4.2 Battery-saving mode interference. In Battery Saving mode, the number of satellites tracked steadily (≤ 1 s between each sample) since the beginning dropped to 2 from 8 throughout the logging period. In particular, the fragmentation of the signal tracking for other satellites occurred mainly during the first 70s, with a position offset reaching ± 10 m from the median. The Figure 15b further demonstrates this, with HDOP peaking at 12. In comparison the standard mode showed an offset of ± 1.5 points. As a result the position point cloud spreads significantly, with a 50% error of 7.6 m against a 50% error of 2.2 m in standard mode.

In this scenario, energy-saving optimizations compromised tracking continuity, signal stability, position initial convergence time and overall accuracy. Even though the receiver held a continuous HW sampling clock, the policies applied by the OS may have affected other parts of the GNSS SW/HW stack.

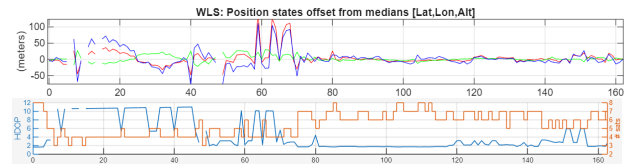


Figure 13: Position states offset from median (top), HDOP and #satellites (bottom) in Battery-saving mode.

5 CONCLUSION

This study highlights GNSS vulnerability to intentional interference and everyday smartphone usage through controlled experiments and MATLAB-based analysis of raw Android GNSS data:

- **Spoofing attacks** displaced the estimated position by ~ 300 m, generating clear anomalies in calculated speed and positional medians.
- **Timing delays** as small as 0.1 ms introduced significant pseudorange inconsistencies and clock bias errors without directly shifting position.
- **Smartphone conditions** affected accuracy markedly: normal use yielded 9.2 m error, battery-saving mode 17.9 m, and active phone calls 21.2 m.

These findings underscore the need for real-time anomaly detection and resilience mechanisms in critical applications such as autonomous navigation, aviation, and emergency services, where relying solely on raw positioning data is insufficient.